

Fiches de Cours

Master IMA

Axel PIGEON

axel.pigeon@univ-tlse3.fr

Université Toulouse III – Paul Sabatier

Années universitaires 2025–2027

9 septembre 2026

Table des matières

I Apprentissage Automatique	3
1 Fondements, Réseaux Denses et Optimisation	4
1.1 Généralités	4
1.2 Du Perceptron au MLP	6
1.3 Fonctions d'Activation et Classification Multiclasse	8
1.4 Fonctions de Perte (<i>Loss Functions</i>)	9
1.5 Rétropropagation du Gradient et Dérivation Automatique	9
2 Réseaux Convolutifs	12
2.1 L'Opérateur de Convolution en Vision Artificielle	12
2.2 Sous-Échantillonnage, Champ Réceptif et Calcul	13
2.3 Techniques de Normalisation et Réseaux Résiduels	14
3 Réseaux Récurrents	16
3.1 Modélisation Séquentielle et Réseaux Récurrents (RNN)	16
3.2 Rétropropagation à travers le Temps (<i>BPTT</i>) et Pathologies	17
3.3 Cellules à Portes de Contrôle : LSTM et GRU	17
3.4 Modèles Encodeur-Décodeur et Mécanismes d'Attention	18
4 Modèles de Markov Caché	20
4.1 Modèles de Langage et Hypothèses Markoviennes	20
4.2 Formalisme des Modèles de Markov Cachés (HMM)	22

Apprentissage Automatique

Chapitre 1

Fondements, Réseaux Denses et Optimisation

Contents

1.1	Généralités	4
1.1.1	Un peu d'histoire...	4
1.1.2	Définition et terminologie	5
1.1.3	L'apprentissage	6
1.2	Du Perceptron au MLP	6
1.2.1	Concept	6
1.2.2	Apprentissage	7
1.2.3	Perceptron Multi-Couches (MLP)	7
1.3	Fonctions d'Activation et Classification Multiclasse	8
1.3.1	Fonctions d'activation usuelles	8
1.3.2	Classification multiclasse et fonction Softmax	9
1.4	Fonctions de Perte (<i>Loss Functions</i>)	9
1.4.1	Entropie croisée binaire (BCE)	9
1.5	Rétropropagation du Gradient et Dérivation Automatique	9
1.5.1	Rétropropagation sur un neurone formel	10
1.5.2	Rétropropagation dans un MLP (<i>Backpropagation</i>)	10
1.5.3	Graphes de calcul et implémentation PyTorch	10
1.5.4	Optimisation par descente de gradient avec inertie (Momentum)	11

1.1 Généralités

1.1.1 Un peu d'histoire...

Nous sommes en été 1955 aux États-Unis. La naissance de l'informatique pousse les mathématiciens John McCarthy, Marvin Minsky, Nathaniel Rochester et l'ingénieur Claude Shannon à proposer un projet de recherche sur l'intelligence artificielle au Dartmouth college "A proposal for the Dartmouth summer research project on artificial intelligence". Ils obtiennent ainsi 7500 \$ de la fonction Rockefeller pour organiser un atelier de travail sur les machines pensantes durant deux mois. Cependant, l'intérêt pour ce domaine de recherche est encore très faible parmi la

communauté scientifique. Les faibles performances des premières machines constituent aussi un important frein au développement de prototypes.

En 1957, Frank Rosenblatt présente le premier réseau de neurones tel que nous les connaissons aujourd'hui appelés *Perceptron*. En 1986, la notion de Perceptron multicouches est explorée et en 1998 Yann LeCun et Yoshua Bengio développent le premier réseau de neurones à convolution à la base de la vision par ordinateur.

La victoire de DeepBlue sur Garry Kasparov aux échecs en 1997 démontre la puissance et le potentiel de ces nouvelles technologies. L'affluence de données disponibles ces 30 dernières années et l'évolution des capacités de calcul des ordinateurs a ouvert la porte à des systèmes de plus en plus performants. Enfin, 2015 marque l'apogée des modèles d'IA dits "classiques" avec la victoire d'AlphaGo sur les champions d'Europe et du monde.

1.1.2 Définition et terminologie

Selon Wikipédia, l'intelligence artificielle se définit comme suit :

"L'intelligence artificielle (IA) est l'ensemble des systèmes informatiques capables d'effectuer des tâches typiquement associées à l'intelligence, telles que l'apprentissage, le raisonnement, la résolution de problèmes, la perception ou la prise de décision."

Le but de l'IA est donc de reproduire numériquement le mode de pensée humaine. Ces concepts ont été, et sont toujours, à la base des réseaux de neurones actuels puisqu'ils sont constitués de neurones organisés en couches permettant de traiter l'information. C'est donc une discipline très vaste dans lequel il est facile de confondre les notions.

L'*apprentissage automatique* en est un sous-domaine qui consiste en la création d'algorithmes capables d'apprendre à partir de données précédemment collectées. Il faut donc distinguer plusieurs phases dans le processus de développement d'un tel modèle :

1. La collecte des données d'entraînement.
2. L'entraînement du modèle.
3. La phase de test des performances du modèle.
4. La mise sur le marché pour une utilisation publique.

Nous traiterons ici les phases 1 et 2 de ce processus. Les autres appartiennent à des domaines différents de l'IA. Pour le développement d'un tel modèle, nous devons donc avoir accès à des **observations** constituées d'exemples ou de données collectés. Ces données sont constituées de deux éléments : les **attributs**, appelés *features*, constituent des représentations numériques des observations et les **étiquettes**, ou *labels*, qui associent à chaque élément une catégorie ou une valeur. L'enjeu est donc d'entraîner un modèle à distinguer les différents attributs pour chaque observation et d'être capable de répondre correctement à la question : "Étant donné une observation, à quelle catégorie appartient-elle?". Pour cela, le jeu de données initial, appelée *dataset*, sera séparé en deux, l'un pour l'entraînement, et l'autre pour le test.

Plus formellement, pour un ensemble $E = \{(x_i, y_i) \mid i \in \llbracket 1, N \rrbracket\}$ de données constitué d'observation X et de données Y , on cherche à apprendre une fonction f telle que :

$$f : \begin{cases} X \longrightarrow Y \\ x_i \longmapsto y_j = f(x_i) \end{cases}$$

et l'on souhaite que $i = j$ pour le plus de cas possibles. C'est-à-dire que notre modèle ne se trompe pas dans la prédiction y_i de la classe de x_i . En fonction de la nature de l'ensemble Y , on distinguera plusieurs types de problèmes. Si Y est un ensemble fini, on parlera de *problème de classification*, si $Y = \{\text{vrai}, \text{faux}\}$, on parlera de *prédiction* et si $Y = \mathbb{R}$, on parlera de régression.

1.1.3 L'apprentissage

L'apprentissage constitue le cœur de ce que nous allons voir ici. Cette phase permet de définir "les bonnes réponses" au modèle. On distingue ainsi différents types d'apprentissages :

- **L'apprentissage supervisé** : on dispose ici de données étiquetées et le modèle doit apprendre à distinguer les classes des observations.
- **L'apprentissage non supervisé** : ici le modèle n'a accès qu'aux observations et doit déterminer lui-même leur classes.
- **L'apprentissage par renforcement** : le modèle est ici très différent, il se représente sous la forme d'un agent devant apprendre une série d'actions en fonction de la configuration de son environnement. Pour cela, à chaque essai, on lui attribue une récompense positive ou négative.

Nous nous intéressons ici à l'apprentissage supervisé.

1.2 Du Perceptron au MLP

Le *Perceptron* peut être vu comme le premier réseau de neurones moderne, bien que sa conception date de 1956. À cette époque, Frank Rosenblatt pense à une "solution à tout" basée sur une représentation simplifiée du cerveau humain. L'idée est de recréer un ensemble de neurones connectés les uns aux autres capables d'"apprendre".

1.2.1 Concept

On se pose donc un problème à deux classes (c_- , c_+) avec m exemples :

$$\mathcal{D} = \{(x_i, y_i) \in \mathbb{R}^p \times \{-1, 1\} \mid j \in \llbracket 1, m \rrbracket\}$$

Pour chaque exemple, x_i représente l'observation et y_i la classe correspondante. On veut développer un modèle capable de déterminer la classe d'une observation brute. On effectue donc une hypothèse forte qui est que les deux classes c_- et c_+ sont séparables par un hyperplan de $\mathbb{R}^p$. Le problème revient donc à construire une famille $(w_1, \dots, w_p)$ de *poids* qui définiront cet hyperplan. Formellement, on veut "apprendre" une fonction y telle que :

$$\forall i \in \llbracket 1, m \rrbracket, \quad y(x) = \begin{cases} 1 & \text{si } \sum_{j=1}^p w_j x_j > T \\ -1 & \text{sinon} \end{cases}.$$

On appellera cette fonction y , notée $\hat{y}$ la *prédiction*. Ici notre réseau de neurones a déjà pris forme. Il se représente encore plutôt sous la forme d'un neurone :

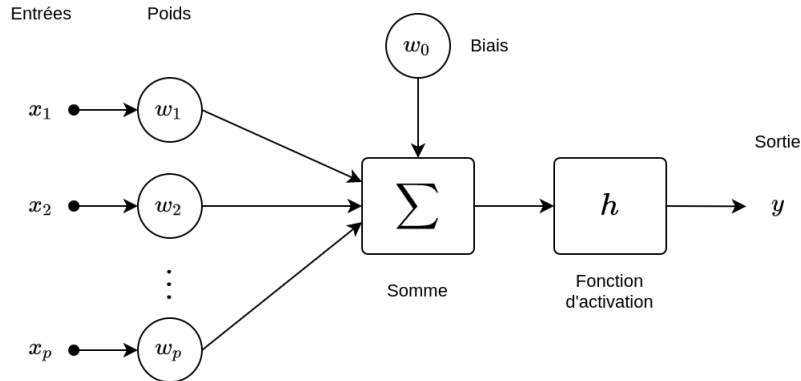


FIGURE 1.1 – Schéma formel d'un neurone

On utilisera aussi une *fonction d'activation* représentée par le seuil T . Plus tard, nous utiliserons des fonctions plus complexes pour donner une valeur de "probabilité" à la sortie d'un neurone.

1.2.2 Apprentissage

Une fois la structure posée, nous devons définir un algorithme pour apprendre les poids $(w_1, \dots, w_p)$ dans le but d'avoir une séparation optimale des deux classes dans $\mathbb{R}^p$. L'idée est donc de faire passer des valeurs x dans le perceptron et d'actualiser les poids w_j si la prédiction $\hat{y}$ est différente de la classe y . Une règle simple d'actualisation est de modifier w_j en fonction de la valeur x_j de l'entrée qu'il pondère. On aura donc la règle suivante :

$$w = \begin{cases} w + x & \text{si } y = 1 \text{ et } \hat{y} = -1 \\ w - x & \text{si } y = -1 \text{ et } \hat{y} = 1 \end{cases} \iff w = w + yx \text{ si } x \text{ est mal classé.}$$

Remarque (Variantes) Notre définition de l'actualisation des poids les modifie à chaque itération lors de l'apprentissage. Cela permet une convergence rapide de l'apprentissage, mais peut conduire à une instabilité si le nombre de poids à traiter est trop grand. Une solution est donc d'effectuer un *apprentissage par batch*. Cela consiste à faire passer successivement plusieurs vecteurs x , on calcule ensuite l'erreur moyenne commise et on met à jour les poids en fonction de cette erreur moyenne. Un autre possibilité est d'utiliser un *learning rate* ν qui bride l'apprentissage. On utilise alors la règle suivante pour actualiser w :

$$w = \eta(y - \hat{y})x.$$

Les perceptrons permettent donc de simuler des fonctions booléennes comme illustré ci-dessous :

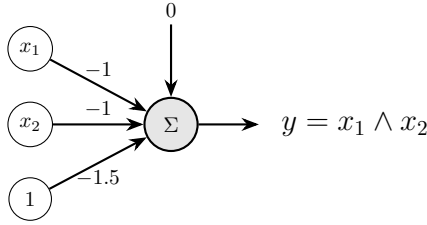


FIGURE 1.2 – Fonction AND avec un perceptron

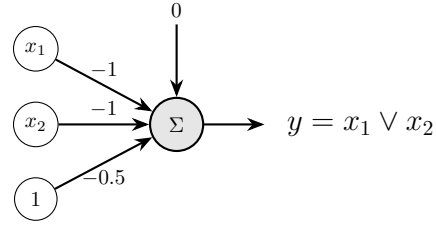


FIGURE 1.3 – Fonction OR avec un perceptron

Après réflexion, on se rend compte qu'il est impossible de simuler la fonction XOR avec un seul neurone. Ce sera donc l'objectif des *MLP (Multi Layer Perceptron)* composés de plusieurs couches de neurones interconnectés.

1.2.3 Perceptron Multi-Couches (MLP)

Comme énoncé précédemment, le perceptron simple de Rosenblatt présente une limitation fondamentale mise en lumière par Marvin Minsky et Seymour Papert en 1969 : l'impossibilité géométrique de séparer des classes qui ne sont pas linéairement séparables dans l'espace d'entrée $\mathbb{R}^d$, à l'instar de la fonction logique XOR (ou exclusif).

Pour contourner ce problème, l'approche naturelle consiste à empiler plusieurs couches de neurones formels. Un *Perceptron Multi-Couches* (Multi-Layer Perceptron, noté MLP) est composé :

1. D'une couche d'entrée (vecteur $x \in \mathbb{R}^d$) ;
2. D'une suite de $L \geq 1$ **couches cachées** (hidden layers), qui apprennent des représentations hiérarchiques distribuées ;
3. D'une couche de sortie produisant la prédiction $\hat{y}$.

Définition (Formalisation de la passe avant d'un MLP) . Soit un réseau à L couches cachées. Notons $h^{(0)} = x \in \mathbb{R}^{d_0}$ le vecteur d'entrée. Pour chaque couche $k \in \llbracket 1, L \rrbracket$ de dimension d_k , le vecteur d'activation est régi par les relations de récurrence :

$$a^{(k)} = W^{(k)}h^{(k-1)} + b^{(k)} \in \mathbb{R}^{d_k} \quad (\text{pré-activation}) \quad (1.2.1)$$

$$h^{(k)} = g^{(k)}\left(a^{(k)}\right) \in \mathbb{R}^{d_k} \quad (\text{activation}) \quad (1.2.2)$$

où $W^{(k)} \in \mathbb{R}^{d_k \times d_{k-1}}$ désigne la matrice de poids de la couche k , $b^{(k)} \in \mathbb{R}^{d_k}$ le vecteur de biais associé, et $g^{(k)} : \mathbb{R} \rightarrow \mathbb{R}$ une fonction d'activation non linéaire appliquée composante par composante. Pour la couche de sortie $k = L + 1$, on applique une fonction terminale o :

$$\hat{y} = h^{(L+1)} = o\left(W^{(L+1)}h^{(L)} + b^{(L+1)}\right). \quad (1.2.3)$$

Remarque (Interprétation géométrique des frontières de décision) Chaque neurone d'une première couche cachée partitionne l'espace $\mathbb{R}^d$ par un demi-espace délimité par un hyperplan affine. Les neurones de la deuxième couche cachée effectuent une intersection booléenne (opérateur ET) de ces demi-espaces, formant ainsi des polyèdres convexes. Enfin, les couches suivantes peuvent réaliser l'union (opérateur OU) de régions convexes disjointes, rendant le réseau théoriquement capable d'approximer n'importe quelle frontière de décision fermée ou ouverte arbitrairement complexe.

1.3 Fonctions d'Activation et Classification Multiclasse

1.3.1 Fonctions d'activation usuelles

Le choix d'une fonction d'activation non linéaire g est indispensable : sans elle, la composition successive d'opérations affines $W^{(L)}(\dots(W^{(1)}x + b^{(1)})\dots) + b^{(L)}$ resterait strictement équivalente à une unique transformation affine globale $\tilde{W}x + \tilde{b}$, ruinant tout l'intérêt de la profondeur.

Définition (Fonctions d'activation courantes) . On définit ainsi différentes fonctions d'activations communément utilisées en sortie d'un neurone. Pour tout réel $z \in \mathbb{R}$ on a :

- **Sigmoïde (logistique)** :

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (1.3.1)$$

Elle projette $\mathbb{R}$ sur $]0, 1[$ et vérifie l'identité remarquable $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

- **Tangente hyperbolique** :

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in]-1, 1[, \quad (1.3.2)$$

centrée en zéro, de dérivée $\tanh'(z) = 1 - \tanh^2(z)$.

- **Rectified Linear Unit (ReLU)** :

$$g(z) = \max(0, z). \quad (1.3.3)$$

Sa dérivée vaut 1 pour $z > 0$ et 0 pour $z < 0$. Elle évite la saturation des gradients pour des entrées positives.

- **Leaky ReLU** : pour $\alpha \in]0, 1[$ (couramment $\alpha = 0,01$), $g(z) = \max(\alpha z, z)$. Elle évite le problème des "neurones morts" lorsque $z < 0$.

- **Softplus :**

$$g(z) = \log(1 + e^z), \quad (1.3.4)$$

approximation analytique différentiable de la ReLU.

1.3.2 Classification multiclass et fonction Softmax

Considérons un problème de classification à $K > 2$ classes mutuellement exclusives. La couche finale produit un vecteur de scores réels non normalisés $a = W^{(L+1)}h^{(L)} + b^{(L+1)} \in \mathbb{R}^K$, appelés *logits*.

Définition (Opérateur Softmax) . L'opérateur Softmax normalise un vecteur de logits $a = (a_1, \dots, a_K)^\top$ en une distribution discrète de probabilités $p = (p_1, \dots, p_K)^\top \in [0, 1]^K$ définie par :

$$\forall c \in \llbracket 1, K \rrbracket, \quad p_c = \text{Softmax}(a)_c = \frac{e^{a_c}}{\sum_{j=1}^K e^{a_j}}. \quad (1.3.5)$$

On a immédiatement $\forall c, p_c > 0$ et $\sum_{c=1}^K p_c = 1$. La classe prédite est $\hat{y} = \arg \max_c p_c$.

1.4 Fonctions de Perte (*Loss Functions*)

En apprentissage supervisé, puisque nous connaissons les *labels* associés à chaque prédiction, nous sommes capables d'évaluer le modèle en temps réel sur chacune de ses prédictions sur les données d'entraînement. Ainsi, l'évaluation de la discordance entre la cible réelle y et la sortie estimée $\hat{y}$ s'effectue au moyen d'une fonction de coût différentiable $\mathcal{L}(y, \hat{y})$ que l'on appellera *fonction de perte* ou *loss*.

1.4.1 Entropie croisée binaire (BCE)

Remarque (Limites de la MSE) Pour une classification binaire avec $y \in \{0, 1\}$ et une prédiction de probabilité $\hat{y} = \sigma(z) \in]0, 1[$, l'erreur quadratique usuelle MSE s'avère sous-optimale du fait de zones de saturation où le gradient s'annule pour des erreurs maximales. On introduit la perte d'entropie croisée binaire.

Définition (Binary Cross-Entropy) . Pour $y \in \{0, 1\}$ et $\hat{y} \in]0, 1[$:

$$\mathcal{L}_{\text{BCE}}(y, \hat{y}) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}). \quad (1.4.1)$$

Proposition (Stabilité numérique de la BCE) En injectant l'activation logistique $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$ directement dans la perte, on évite les dépassements de capacité flottante (*overflow*) via l'expression analytique exacte :

$$\mathcal{L}_{\text{BCE}}(y, \sigma(z)) = \max(z, 0) - z \cdot y + \log(1 + e^{-|z|}). \quad (1.4.2)$$

1.5 Rétropropagation du Gradient et Dérivation Automatique

Comme vu au début de ce chapitre, l'apprentissage se fait par une recherche des "meilleurs" poids possibles pour le modèle dans l'objectif de minimiser l'erreur commise. Maintenant que

nous avons introduit la fonction de perte *loss*, nous cherchons donc à faire une minimisation de cette fonction *loss*.

La principale problématique dans ce processus est le calcul de son *gradient*. En effet, pour trouver des candidats minimiseurs, la condition d'optimalité d'ordre 1 nous demande d'annuler ce gradient. Nous devons donc être capable de calculer les dérivées partielles de la *loss* en fonction des poids du modèle. On utilise pour cela l'algorithme de *rétropropagation* à l'aide de *graphes computationnels*.

1.5.1 Rétropropagation sur un neurone formel

Considérons un unique neurone formel doté d'une entrée $x \in \mathbb{R}^d$, d'une pré-activation $a = w^\top x + b$, d'une activation $\hat{y} = \sigma(a)$, et d'une perte quadratique $\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$. Par application directe de la règle de dérivation en chaîne (*Chain Rule*) :

$$\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{d\hat{y}}{da} \cdot \frac{\partial a}{\partial w_i} = -(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) \cdot x_i. \quad (1.5.1)$$

On retrouve le terme d'erreur $(y - \hat{y})$, pondéré par la pente de la sigmoïde $\hat{y}(1 - \hat{y})$ et la projection d'entrée x_i . Si l'on remplace la perte quadratique par la $\mathcal{L}_{\text{BCE}}$, le terme $\hat{y}(1 - \hat{y})$ se simplifie élégamment au dénominateur :

$$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial a} = \hat{y} - y \implies \frac{\partial \mathcal{L}_{\text{BCE}}}{\partial w} = (\hat{y} - y)x. \quad (1.5.2)$$

1.5.2 Rétropropagation dans un MLP (*Backpropagation*)

Dans un graphe multicouches, les erreurs locales sont propagées de l'aval vers l'amont :

Définition (Rétropropagation multicouche) . Posons $\delta^{(k)} = \frac{\partial \mathcal{L}}{\partial a^{(k)}}$ le vecteur de sensibilité de la couche k .

$$\text{Couche de sortie } (L + 1) : \quad \delta^{(L+1)} = \nabla_{a^{(L+1)}} \mathcal{L} \quad (1.5.3)$$

$$\text{Couches cachées } (k = L, \dots, 1) : \quad \delta^{(k)} = \left((W^{(k+1)})^\top \delta^{(k+1)} \right) \odot g'^{(k)} \left(a^{(k)} \right) \quad (1.5.4)$$

où $\odot$ désigne le produit d'Hadamard (terme à terme). Les gradients s'en déduisent immédiatement :

$$\frac{\partial \mathcal{L}}{\partial W^{(k)}} = \delta^{(k)} \left(h^{(k-1)} \right)^\top \in \mathbb{R}^{d_k \times d_{k-1}}, \quad \frac{\partial \mathcal{L}}{\partial b^{(k)}} = \delta^{(k)} \in \mathbb{R}^{d_k}. \quad (1.5.5)$$

1.5.3 Graphes de calcul et implémentation PyTorch

En pratique, le calcul symbolique manuel des dérivées est remplacé par l'auto-différenciation en mode inverse (*reverse-mode automatic differentiation*) matérialisée par un graphe acyclique orienté (DAG).

```

1 import torch
2 import torch.nn as nn
3
4 class MultiLayerPerceptron(nn.Module):
5     def __init__(self, in_features: int, hidden_dim: int, out_classes: int):
6         super(MultiLayerPerceptron, self).__init__()
7         self.network = nn.Sequential(
8             nn.Linear(in_features, hidden_dim),
9             nn.ReLU(),
10            nn.Linear(hidden_dim, hidden_dim),

```

```

11         nn.ReLU(),
12         nn.Linear(hidden_dim, out_classes)
13     )
14
15     def forward(self, x: torch.Tensor) -> torch.Tensor:
16         # Retourne les logits bruts (non normalises)
17         return self.network(x)
18
19 # Instanciación et passe avant/arrière
20 model = MultilayerPerceptron(in_features=784, hidden_dim=128, out_classes=10)
21 criterion = nn.CrossEntropyLoss() # Integre log-softmax et NLL
22 optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
23
24 x_dummy = torch.randn(32, 784) # Batch de 32 observations
25 y_target = torch.randint(0, 10, (32,))
26
27 optimizer.zero_grad()
28 logits = model(x_dummy)
29 loss = criterion(logits, y_target)
30 loss.backward() # Calcul automatique des gradients via Autograd
31 optimizer.step()

```

Listing 1.1 – Implémentation d'un MLP complet sous PyTorch

1.5.4 Optimisation par descente de gradient avec inertie (Momentum)

La descente de gradient stochastique standard (SGD) présente de fortes oscillations transverses dans les vallées mal conditionnées (matrice hessienne à fort conditionnement). La méthode avec moment régularise la trajectoire en accumulant une moyenne exponentielle glissante des vitesses passées :

$$v_t = \beta v_{t-1} + \alpha \nabla_W \mathcal{L}_t \quad \text{avec } \beta \in [0, 1[\quad (1.5.6)$$

$$W_{t+1} = W_t - v_t \quad (1.5.7)$$

où $\alpha > 0$ représente le pas d'apprentissage (*learning rate*).

Chapitre 2

Réseaux Convolutifs

Contents

2.1 L'Opérateur de Convolution en Vision Artificielle	12
2.1.1 Formulation mathématique	13
2.1.2 Dimensions, Stride, Padding et Dilatation	13
2.2 Sous-Échantillonnage, Champ Réceptif et Calcul	13
2.2.1 Couches de Pooling	14
2.2.2 Notion de champ réceptif (<i>Receptive Field</i>)	14
2.2.3 Algorithmique Bas Niveau : L'Approche Im2Col et GEMM	14
2.3 Techniques de Normalisation et Réseaux Résiduels	14
2.3.1 Normalisation par Lots (<i>Batch Normalization</i>)	14
2.3.2 Réseaux Résiduels (ResNet)	15
2.3.3 Implémentation d'un bloc résiduel sous PyTorch	15

Dans le cadre du traitement d'images numériques, l'utilisation de couches entièrement connectées (MLP) devient vite impraticable :

1. **Explosion du nombre de paramètres** : pour une image couleur haute définition de taille $1000 \times 1000 \times 3$, un premier neurone connecté à chaque pixel requiert 3×10^6 poids, induisant un coût mémoire prohibitif et un sur-apprentissage critique.
2. **Perte de corrélation spatiale** : vectoriser une image 2D/3D détruit la proximité géométrique locale entre pixels adjacents.
3. **Absence d'invariance par translation** : si un motif d'intérêt (un œil, une roue) se déplace dans l'image, un MLP doit réapprendre ce motif sur chacun des emplacements distincts.

La solution réside dans l'exploitation de la structure spatiale via deux principes directeurs : la *connectivité locale* (chaque neurone n'observe qu'un sous-voisinage) et le *partage des paramètres* (weight sharing).

2.1 L'Opérateur de Convolution en Vision Artificielle

L'objectif est donc d'être capable d'identifier des schémas caractéristiques dans une image, indépendamment de sa position dans celle-ci. Pour cela, on utilisera un opérateur de *convolution*. L'idée est de balayer la surface de l'image par une matrice plus petite appelée *noyau*. À chaque déplacement, on effectuera le produit de la partie de l'image couverte par le noyau. L'image ainsi obtenue sera plus petite et contiendra la même information mais sous forme condensée.

En effectuant cette étape de façon successive, on sera capable de réduire la taille de l'image d'origine à celle d'un vecteur qui pourra être facilement interprété par un MLP.

2.1.1 Formulation mathématique

En traitement du signal numérique, l'opération discrète bidimensionnelle appliquée entre une entrée matricielle I et un noyau (*kernel*) $K \in \mathbb{R}^{k_h \times k_w}$ est définie usuellement sous la forme d'une corrélation croisée :

$$S(i, j) = (I * K)(i, j) = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} I(i+m, j+n) K(m, n). \quad (2.1.1)$$

Dans une couche de convolution standard d'un CNN, l'entrée est un tenseur à trois dimensions $\mathcal{X} \in \mathbb{R}^{C_{\text{in}} \times H_{\text{in}} \times W_{\text{in}}}$ où C_{in} est le nombre de canaux (ex : Rouge, Vert, Bleu). La couche est paramétrée par C_{out} filtres tridimensionnels $W_k \in \mathbb{R}^{C_{\text{in}} \times k_h \times k_w}$ et un vecteur de biais $b \in \mathbb{R}^{C_{\text{out}}}$. La k -ième carte de caractéristiques de sortie s'exprime par :

$$S_k = b_k + \sum_{c=1}^{C_{\text{in}}} \mathcal{X}_c * W_{k,c}. \quad (2.1.2)$$

2.1.2 Dimensions, Stride, Padding et Dilatation

Propriété (Régression des dimensions spatiales) . Soit une entrée de taille $(H_{\text{in}}, W_{\text{in}})$, convoluée avec un noyau de taille (K_h, K_w) , un rembourrage (*padding*) P , un pas (*stride*) S , et un taux de dilatation (*dilation*) D . La taille de la carte de sortie $(H_{\text{out}}, W_{\text{out}})$ vérifie :

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P_h - D_h(K_h - 1) - 1}{S_h} + 1 \right\rfloor, \quad (2.1.3)$$

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P_w - D_w(K_w - 1) - 1}{S_w} + 1 \right\rfloor. \quad (2.1.4)$$

Proposition (Nombre de paramètres d'une couche convolutive) Pour une couche convolutive associant C_{in} canaux d'entrée à C_{out} canaux de sortie au travers de noyaux $K_h \times K_w$, le nombre de paramètres scalaires libres vaut :

$$N_{\text{params}} = C_{\text{out}} \times (C_{\text{in}} \times K_h \times K_w + 1) \quad (2.1.5)$$

où le terme $+1$ comptabilise le biais scalaire associé à chaque carte de sortie. Ce nombre est strictement indépendant de la résolution spatiale $(H_{\text{in}}, W_{\text{in}})$ de l'image.

Remarque Les couches de convolutions servent donc à *extraire* des schémas (*pattern*) et les caractéristiques (*features*) de l'image et de les compresser en vecteurs. Ensuite, un MLP est chargé d'*interpréter* ces *features* en prédiction (voir chapitre précédent). L'utilisation des convolutions permet de réduire drastiquement le nombre de paramètres et de couches à utiliser pour l'extraction de ces *features*.

2.2 Sous-Échantillonnage, Champ Réceptif et Calcul

L'opération de convolution est très simple à mettre en place algorithmiquement. En fonction de l'opération effectuée avec son résultat et de la dimension de son noyau, on peut observer plusieurs propriétés remarquables. Elles permettent ensuite de définir de nouvelles couches de convolutions ayant différents cas d'usages.

2.2.1 Couches de Pooling

Les couches de regroupement (*pooling*) réduisent la résolution spatiale des tenseurs intermédiaires pour accroître l'invariance aux translations locales et diminuer la charge computationnelle :

- **Max-Pooling** : extrait la valeur maximale sur une fenêtre $p_h \times p_w$. Il conserve le signal d'activation dominant tout en rejetant les variations de fond.
- **Average-Pooling** : calcule la moyenne arithmétique sur la fenêtre, agissant comme un filtre passe-bas régularisant.

2.2.2 Notion de champ réceptif (*Receptive Field*)

Le champ réceptif d'un neurone correspond à la zone d'extension dans l'image d'entrée d'origine qui influence directement son activation.

Proposition (Croissance du champ réceptif) Pour une succession de couches convolutives à noyaux de taille k_l et de pas unitaire, le champ réceptif cumulé R_L à la couche L vérifie :

$$R_L = R_{L-1} + (k_L - 1) = 1 + \sum_{l=1}^L (k_l - 1). \quad (2.2.1)$$

Ainsi, l'empilement de deux convolutions successives de taille 3×3 possède un champ réceptif équivalent de 5×5 , tout en utilisant $2 \times (3^2) = 18$ poids au lieu de $5^2 = 25$ pour un noyau direct, tout en introduisant deux non-linéarités intermédiaires au lieu d'une.

2.2.3 Algorithmique Bas Niveau : L'Approche Im2Col et GEMM

Les unités graphiques de calcul (GPU) n'exécutent pas les convolutions spatiales par boucles imbriquées, mais projettent l'opération sous forme de multiplication matricielle dense hautement vectorisée (*General Matrix Multiply*, GEMM) :

1. **L'algorithme Im2Col** déplie chaque bloc spatial réceptif $C_{in} \times K_h \times K_w$ présent dans le tenseur d'entrée en un vecteur colonne, formant une vaste matrice :

$$X_{col} \in \mathbb{R}^{(C_{in} \cdot K_h \cdot K_w) \times (H_{out} \cdot W_{out})}. \quad (2.2.2)$$

2. Les poids des filtres sont aplatis en lignes pour construire une matrice :

$$W_{row} \in \mathbb{R}^{C_{out} \times (C_{in} \cdot K_h \cdot K_w)}. \quad (2.2.3)$$

3. Le produit matriciel dense calcule l'ensemble des réponses en une passe :

$$Y_{flat} = W_{row} \cdot X_{col} \in \mathbb{R}^{C_{out} \times (H_{out} \cdot W_{out})}. \quad (2.2.4)$$

4. La matrice résultante est réorganisée (*reshapée*) en dimension $(C_{out}, H_{out}, W_{out})$.

2.3 Techniques de Normalisation et Réseaux Résiduels

2.3.1 Normalisation par Lots (*Batch Normalization*)

Pour stabiliser la covariance interne des couches profondes (*Internal Covariate Shift*), on introduit la couche de Batch Normalization.

Définition (Batch Normalization) . Pour un mini-lot d’activations scalaires $\mathcal{B} = \{x_1, \dots, x_m\}$ correspondant à un canal donné :

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m x_i, \quad \sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad (2.3.1)$$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad (2.3.2)$$

$$y_i = \gamma \hat{x}_i + \beta \quad \text{avec } \gamma, \beta \text{ paramètres différentiables appris.} \quad (2.3.3)$$

2.3.2 Réseaux Résiduels (ResNet)

Au-delà d’une vingtaine de couches, l’entraînement d’un réseau standard subit une dégradation critique due à l’évanescence des gradients. He et al. (2015) introduisent les connexions résiduelles directes (*skip connections*) :

$$\mathcal{H}(x) = \mathcal{F}(x, \{W_i\}) + x \quad (2.3.4)$$

où $\mathcal{F}$ représente la transformation non linéaire de quelques couches convolutives. Si la solution optimale est l’identité, le réseau n’a besoin que d’annuler les poids $\mathcal{F}(x) \rightarrow 0$, ce qui est infiniment plus aisé à apprendre. Lors de la rétropropagation, le gradient transite sans atténuation multiplicative via la dérivation directe :

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial \mathcal{H}} \left(\frac{\partial \mathcal{F}}{\partial x} + I \right). \quad (2.3.5)$$

2.3.3 Implémentation d’un bloc résiduel sous PyTorch

```

1 import torch
2 import torch.nn as nn
3
4 class ResidualBlock(nn.Module):
5     def __init__(self, channels: int):
6         super(ResidualBlock, self).__init__()
7         self.conv_block = nn.Sequential(
8             nn.Conv2d(channels, channels, kernel_size=3, stride=1, padding=1,
9             bias=False),
10            nn.BatchNorm2d(channels),
11            nn.ReLU(inplace=True),
12            nn.Conv2d(channels, channels, kernel_size=3, stride=1, padding=1,
13            bias=False),
14            nn.BatchNorm2d(channels)
15        )
16        self.relu = nn.ReLU(inplace=True)
17
18    def forward(self, x: torch.Tensor) -> torch.Tensor:
19        identity = x
20        out = self.conv_block(x)
21        out += identity # Connexion résiduelle directe
22        out = self.relu(out)
23        return out

```

Listing 2.1 – Bloc résiduel classique sous PyTorch

Chapitre 3

Réseaux Récurrents

Contents

3.1	Modélisation Séquentielle et Réseaux Récurrents (RNN)	16
3.1.1	Architecture et Dépliage Temporel	16
3.2	Rétropropagation à travers le Temps (BPTT) et Pathologies	17
3.2.1	Calcul du gradient BPTT	17
3.2.2	Évanescence et explosion du gradient	17
3.3	Cellules à Portes de Contrôle : LSTM et GRU	17
3.3.1	Long Short-Term Memory (LSTM)	17
3.3.2	Gated Recurrent Unit (GRU)	18
3.4	Modèles Encodeur-Décodeur et Mécanismes d'Attention	18
3.4.1	Le paradigme Sequence-to-Sequence (Seq2Seq)	18
3.4.2	Mécanisme d'attention d'alignement dynamique	18
3.4.3	Implémentation PyTorch d'un module Seq2Seq récurrent	18

3.1 Modélisation Séquentielle et Réseaux Récurrents (RNN)

Contrairement aux images dont les dimensions peuvent être ajustées à des formats fixes, les signaux temporels, acoustiques ou textuels se présentent sous la forme de séquences de longueur variable :

$$X = (x^{(1)}, x^{(2)}, \dots, x^{(T)}), \quad x^{(t)} \in \mathbb{R}^d. \quad (3.1.1)$$

3.1.1 Architecture et Dépliage Temporel

Un réseau récurrent traite séquentiellement chaque élément en maintenant un vecteur d'état interne $h^{(t)} \in \mathbb{R}^m$, constituant la mémoire synthétique du passé.

Définition (Équations du RNN simple d'Elman) . Pour chaque pas temporel $t \in \llbracket 1, T \rrbracket$:

$$h^{(t)} = \tanh(W h^{(t-1)} + U x^{(t)} + b_h) \quad (3.1.2)$$

$$\hat{y}^{(t)} = \text{Softmax}(V h^{(t)} + b_y) \quad (3.1.3)$$

où $U \in \mathbb{R}^{m \times d}$ est la matrice de projection des entrées, $W \in \mathbb{R}^{m \times m}$ la matrice de récurrence

état-à-état, et $V \in \mathbb{R}^{K \times m}$ la matrice de décodage vers l'espace de sortie. Les paramètres $\{U, W, V, b_h, b_y\}$ sont **partagés** à travers tous les pas de temps.

3.2 Rétropropagation à travers le Temps (*BPTT*) et Pathologies

3.2.1 Calcul du gradient BPTT

L'erreur globale commise sur la séquence est la somme des coûts temporels $E = \sum_{t=1}^T E_t$. Pour dériver le gradient par rapport à la matrice de récurrence W , on déroule la dépendance temporelle :

$$\frac{\partial E}{\partial W} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial E_t}{\partial h^{(t)}} \cdot \left(\prod_{j=k+1}^t \frac{\partial h^{(j)}}{\partial h^{(j-1)}} \right) \cdot \frac{\partial h^{(k)}}{\partial W}. \quad (3.2.1)$$

3.2.2 Évanescence et explosion du gradient

Le produit matriciel cumulé $\prod_{j=k+1}^t \frac{\partial h^{(j)}}{\partial h^{(j-1)}}$ gouverne la propagation temporelle du signal d'erreur. Si la matrice jacobienne présente des valeurs singulières strictement supérieures à 1, la norme du gradient diverge exponentiellement vers l'infini pour de grandes séquences : c'est l'**explosion du gradient**. On y remédie par la troncature heuristique (*Gradient Clipping*) :

$$g \leftarrow g \cdot \min \left(1, \frac{\text{seuil}}{\|g\|_2} \right). \quad (3.2.2)$$

Inversement, si les valeurs propres sont inférieures à 1, le gradient s'évanouit exponentiellement vers zéro au fur et à mesure que $t - k$ croît (**dissipation du gradient**), interdisant l'apprentissage de corrélations à long terme.

3.3 Cellules à Portes de Contrôle : LSTM et GRU

3.3.1 Long Short-Term Memory (LSTM)

Conçue par Hochreiter et Schmidhuber (1997), l'architecture LSTM résout l'évanescence du gradient via un état cellulaire interne C_t mis à jour par des opérations additives protégées par des portes logiques différentiables.

Définition (Formulation complète d'un bloc LSTM) . Soit x_t l'entrée au temps t et h_{t-1} l'état caché antérieur. Les portes sont régies par :

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{porte d'oubli / Forget Gate}) \quad (3.3.1)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{porte d'entrée / Input Gate}) \quad (3.3.2)$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C) \quad (\text{état candidat}) \quad (3.3.3)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (\text{mise à jour cellulaire additive}) \quad (3.3.4)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (\text{porte de sortie / Output Gate}) \quad (3.3.5)$$

$$h_t = o_t \odot \tanh(C_t) \quad (\text{état caché final}). \quad (3.3.6)$$

Grâce à la relation linéaire $C_t = f_t \odot C_{t-1} + \dots$, si la porte d'oubli est activée à saturation ($f_t \approx 1$), le gradient d'erreur se propage à travers l'axe temporel sans aucune atténuation multiplicative exponentielle.

3.3.2 Gated Recurrent Unit (GRU)

Proposé par Cho et al. (2014), le bloc GRU fusionne l'état cellulaire et l'état caché tout en réduisant le nombre de portes à deux :

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z) \quad (\text{porte de mise à jour / Update Gate}) \quad (3.3.7)$$

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r) \quad (\text{porte de réinitialisation / Reset Gate}) \quad (3.3.8)$$

$$\tilde{h}_t = \tanh(W[r_t \odot h_{t-1}, x_t] + b) \quad (3.3.9)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \quad (3.3.10)$$

3.4 Modèles Encodeur-Décodeur et Mécanismes d'Attention

3.4.1 Le paradigme Sequence-to-Sequence (Seq2Seq)

Dans une tâche de traduction ou de résumé, la longueur de la séquence générée est distincte de la séquence source. L'encodeur compresse l'intégralité de la séquence d'entrée en un vecteur contexte unique $c = h^{(T_{in})}$. Le décodeur génère les sorties mot par mot en conditionnant chaque tirage sur ce vecteur. Ce goulot d'étranglement unique entraîne une forte détérioration des performances lorsque la phrase dépasse une vingtaine de mots.

3.4.2 Mécanisme d'attention d'alignement dynamique

L'attention (Bahdanau et al., 2015 ; Luong et al., 2015) lève cette contrainte en permettant au décodeur de consulter l'ensemble des états intermédiaires de l'encodeur $(s_1, \dots, s_S)$ à chaque pas de génération t .

Définition (Calcul du vecteur de contexte par attention) . Pour un état courant du décodeur h_t et un état source s_k :

$$\text{Score d'alignement : } e_{t,k} = \begin{cases} v_a^\top \tanh(W_a h_t + U_a s_k) & (\text{Attention additive de Bahdanau}) \\ h_t^\top W_a s_k & (\text{Attention générale de Luong}) \\ h_t^\top s_k & (\text{Attention par produit scalaire direct}) \end{cases} \quad (3.4.1)$$

$$\text{Poids d'attention : } \alpha_{t,k} = \frac{\exp(e_{t,k})}{\sum_{j=1}^S \exp(e_{t,j})} \in [0, 1] \quad (3.4.2)$$

$$\text{Vecteur de contexte : } c_t = \sum_{k=1}^S \alpha_{t,k} s_k. \quad (3.4.3)$$

3.4.3 Implémentation PyTorch d'un module Seq2Seq récurrent

```
1 import torch
2 import torch.nn as nn
3
4 class Seq2SeqRNN(nn.Module):
5     def __init__(self, vocab_size: int, embed_dim: int, hidden_dim: int):
6         super(Seq2SeqRNN, self).__init__()
7         self.embedding = nn.Embedding(vocab_size, embed_dim)
8         self.encoder = nn.LSTM(embed_dim, hidden_dim, batch_first=True,
9                                 bidirectional=True)
```

```
9         # Decodeur prenant en compte la bidirectionnalite (hidden_dim * 2)
10         self.decoder = nn.LSTM(embed_dim, hidden_dim * 2, batch_first=True)
11         self.fc_out = nn.Linear(hidden_dim * 2, vocab_size)
12
13     def forward(self, src: torch.Tensor, trg: torch.Tensor) -> torch.Tensor:
14         # Encodage
15         embedded_src = self.embedding(src)
16         enc_outputs, (h_n, c_n) = self.encoder(embedded_src)
17
18         # Concatener les etats terminaux des deux directions de l'encodeur
19         h_dec = torch.cat((h_n[0:1], h_n[1:2]), dim=2)
20         c_dec = torch.cat((c_n[0:1], c_n[1:2]), dim=2)
21
22         # Decodage autoregressif
23         embedded_trg = self.embedding(trg)
24         dec_outputs, _ = self.decoder(embedded_trg, (h_dec, c_dec))
25         logits = self.fc_out(dec_outputs)
26         return logits
```

Listing 3.1 – Encodeur LSTM et Décodeur avec projection sous PyTorch

Chapitre 4

Modèles de Markov Caché

Contents

4.1	Modèles de Langage et Hypothèses Markoviennes	20
4.1.1	Modélisation séquentielle par règle de la chaîne	20
4.1.2	Du Sac de Mots au Modèle de Markov d'ordre m	21
4.1.3	Modèles à classes de Brown	22
4.2	Formalisme des Modèles de Markov Cachés (HMM)	22
4.2.1	Définition mathématique	22
4.2.2	Les trois problèmes fondamentaux de Rabiner (1989)	23

Ce chapitre traite des *Modèles de Markov Cachés* (Hidden Markov Models, HMM). Historiquement développés pour le traitement automatique de la parole (*Automatic Speech Recognition*, Rabiner, 1989) et le traitement du langage naturel (étiquetage morpho-syntaxique *POS tagging*, modèles de langage, alignement bilingue), les HMM offrent un cadre probabiliste rigoureux pour les séquences discrètes. Bien que les modèles de fondation neuronaux (Transformers) dominent aujourd'hui ces applications, les algorithmes sous-jacents aux HMM (Forward-Backward, Viterbi, Baum-Welch/EM) constituent les bases fondamentales des modèles graphiques probabilistes et de la programmation dynamique.

4.1 Modèles de Langage et Hypothèses Markoviennes

4.1.1 Modélisation séquentielle par règle de la chaîne

Soit $\mathcal{V}$ un vocabulaire fini de symboles (mots ou *tokens*). On cherche à définir une loi de probabilité conjointe sur des séquences de longueur arbitraire bordées par deux balises déterministes start et stop :

$$O = (\text{start}, O_1, O_2, \dots, O_n, \text{stop}) \in \mathcal{V}^*. \quad (4.1.1)$$

Par application exacte de la règle de décomposition en chaîne des probabilités conditionnelles, toute distribution sur une séquence s'écrit de manière causale (contexte gauche) :

$$\mathbb{P}(\text{start}, O_1, \dots, O_n, \text{stop}) = \prod_{i=1}^{n+1} \mathbb{P}(O_i \mid \text{start}, O_1, \dots, O_{i-1}) \quad (4.1.2)$$

en posant conventionnellement $O_{n+1} = \text{stop}$.

Remarque (Intuition des dés de Noah Smith et explosion combinatoire) On peut interpréter ce processus génératif à l'aide d'une collection de dés dont les faces représentent les mots de $\mathcal{V}$. À chaque étape, l'historique complet $h = (\text{start}, O_1, \dots, O_{i-1})$ détermine le dé à lancer, dont la distribution sur les faces fournit le mot suivant O_i . Bien que d'une expressivité totale, ce modèle présente deux écueils rédhibitoires :

1. **Explosion paramétrique** : pour un historique de longueur au plus n et un vocabulaire de taille $V = |\mathcal{V}|$, le nombre d'historiques possibles vaut $\sum_{k=0}^{n-1} V^k = \frac{V^n - 1}{V - 1}$. Pour $V = 10^5$ et $n = 10$, cela exigerait de stocker les paramètres d'environ 10^{50} distributions catégorielles, ce qui est matériellement impossible.
2. **Défaut de généralisation** : toute séquence contenant un préfixe non observé lors de l'apprentissage se voit affecter une probabilité nulle.

4.1.2 Du Sac de Mots au Modèle de Markov d'ordre m

Pour rendre l'estimation calculable, on réduit l'historique conditionnel via des hypothèses d'indépendance conditionnelle.

Définition (Modèle Sac de Mots / Unigramme) . L'hypothèse la plus forte pose l'indépendance mutuelle totale de chaque observation vis-à-vis du contexte passé :

$$\mathbb{P}(\text{start}, O_1, \dots, O_n, \text{stop}) = \prod_{i=1}^n \mathbb{P}(O_i). \quad (4.1.3)$$

Ce modèle ne requiert que $V - 1$ paramètres libres. En revanche, il détruit toute syntaxe : la probabilité d'une séquence est invariante par permutation de ses termes.

Le compromis canonique consiste à borner la mémoire du processus aux m derniers symboles émis.

Définition (Chaîne de Markov d'ordre m / n -grammes) . Un modèle de Markov stationnaire d'ordre $m \geq 1$ pose que la loi conditionnelle du token courant ne dépend que des m tokens immédiatement antérieurs :

$$\mathbb{P}(O_i \mid \text{start}, O_1, \dots, O_{i-1}) = \mathbb{P}(O_i \mid O_{i-m}, \dots, O_{i-1}). \quad (4.1.4)$$

La probabilité conjointe de la séquence s'écrit alors :

$$\mathbb{P}(\text{start}, O_1, \dots, O_n, \text{stop}) = \prod_{i=1}^{n+1} \mathbb{P}(O_i \mid O_{i-m}^{i-1}) \quad (4.1.5)$$

où $O_{i-m}^{i-1} = (O_{i-m}, \dots, O_{i-1})$.

Remarque (Taxonomie et complexité des n -grammes) Dans la terminologie du traitement automatique du langage :

- $m = 0$ correspond au modèle **unigramme** ($V - 1$ paramètres) ;
- $m = 1$ correspond au modèle **bigramme** ($V(V - 1)$ paramètres de transition) ;
- $m = 2$ correspond au modèle **trigramme** ($V^2(V - 1)$ paramètres).

Au-delà de $m = 3$ ou 4 , la matrice des comptes devient extrêmement creuse (*sparsity*), rendant indispensables des techniques de lissage statistique (Good-Turing, Kneser-Ney).

4.1.3 Modèles à classes de Brown

Pour injecter de la régularité syntaxique sans subir l'explosion combinatoire des n -grammes lexicaux, Brown et al. (1992) partitionnent le vocabulaire $\mathcal{V}$ en $C \ll V$ classes syntaxiques disjointes via une application déterministe $\text{cl} : \mathcal{V} \rightarrow \llbracket 1, C \rrbracket$ obtenue par partitionnement statistique :

Définition (Modèle de séquence fondé sur des classes) . La génération d'une observation O_i est scindée en deux lois : une transition d'état à état entre classes, suivie de l'émission du mot conditionnellement à sa propre classe :

$$\mathbb{P}(\text{start}, O_1, \dots, O_n, \text{stop}) = \prod_{i=1}^{n+1} \mathbb{P}(O_i \mid \text{cl}(O_i)) \cdot \mathbb{P}(\text{cl}(O_i) \mid \text{cl}(O_{i-1})). \quad (4.1.6)$$

En notant $A \in \mathbb{R}^{C \times C}$ la matrice de transition inter-classes et $B \in \mathbb{R}^{C \times V}$ la matrice d'émission des mots sachant la classe, ce modèle ne requiert que $C^2 + C \times V$ paramètres.

Cependant, ce modèle suppose que la fonction de projection cl est déterministe et connue à l'avance. Or, dans la langue naturelle, un mot est fondamentalement polysémique : le mot « *flies* » peut être un verbe ou un nom selon le contexte. L'état syntaxique sous-jacent est donc par essence **non observé** : c'est l'acte de naissance des *Modèles de Markov Cachés*.

4.2 Formalisme des Modèles de Markov Cachés (HMM)

4.2.1 Définition mathématique

Un modèle de Markov caché de premier ordre modélise un processus bivarié $(S_t, O_t)_{t \geq 1}$ où la suite des états (S_t) forme une chaîne de Markov non observable (latente), et (O_t) est la suite des observations visibles émises à chaque instant conditionnellement à l'état courant.

Définition (Paramètres canoniques d'un HMM) . Un HMM à états discrets est formellement caractérisé par le triplet $\lambda = (\pi, A, B)$:

1. Un ensemble fini d'états cachés $\mathcal{S} = \{s_1, \dots, s_N\}$.
2. Un alphabet fini d'observations $\mathcal{V} = \{v_1, \dots, v_M\}$.
3. La distribution initiale des états $\pi = (\pi_i)_{i=1}^N$ avec $\pi_i = \mathbb{P}(S_1 = s_i)$.
4. La matrice de transition $A = (a_{ij}) \in \mathbb{R}^{N \times N}$ régie par l'hypothèse markovienne :

$$a_{ij} = \mathbb{P}(S_{t+1} = s_j \mid S_t = s_i), \quad \sum_{j=1}^N a_{ij} = 1. \quad (4.2.1)$$

5. La matrice d'émission $B = (b_j(k)) \in \mathbb{R}^{N \times M}$ vérifiant l'hypothèse d'indépendance conditionnelle :

$$b_j(k) = \mathbb{P}(O_t = v_k \mid S_t = s_j), \quad \sum_{k=1}^M b_j(k) = 1. \quad (4.2.2)$$

Pour une séquence d'états $S = (S_1, \dots, S_T)$ et d'observations $O = (O_1, \dots, O_T)$, la loi conjointe se factorise immédiatement grâce aux indépendances conditionnelles du graphe :

$$\mathbb{P}(O, S \mid \lambda) = \pi_{S_1} b_{S_1}(O_1) \prod_{t=2}^T a_{S_{t-1}, S_t} b_{S_t}(O_t). \quad (4.2.3)$$

4.2.2 Les trois problèmes fondamentaux de Rabiner (1989)

